

IFN647 Week 7 Review Questions

Solutions

Q1 (Mutual Information)

For two words (or terms) a and b , their mutual information measure (MIM) is defined as

$$\log \frac{P(a, b)}{P(a)P(b)}$$

where $P(a)$ is the probability that word a occurs in a text window of a given size, $P(b)$ is the probability that word b occurs in a text window, and $P(a, b)$ is the probability that a and b occur in the same text window.

Assume a function $df(docs, a)$ is defined to calculate the number of documents containing word a , and function $df2(docs, a, b)$ returns the number of documents containing both words a and b , where $docs$ is a set of documents.

- (1) Let D be a set of documents. For any two words a and b , use the MIM equation and the provided two functions to calculate their MIM .
- (2) Assume the collection of documents is represented as a dictionary $\{docID: \{term: counts, \dots\}, \dots\}$, for example, $docs = \{D1: \{'term1':3, 'term4':5, 'term5':7\}, D2: \{'term1':5, 'term2':3, 'term3':4, 'term4':6\}, D3: \{'term3':5, 'term4':4, 'term5':6\}, D4: \{'term1':9, 'term4':1, 'term5':2\}, D5: \{'term2':1, 'term4':3, 'term5':2\}, D6: \{'term1':3, 'term3':2, 'term4':4, 'term5':4\}\}$. Define a python function $df2(docs, a, b)$ to calculate n_{ab} - the number of documents that include both terms a and b .
- (3) Assume python function $c_df(docs)$ calculates df value for all terms in $docs$ and returns a $\{term:df, \dots\}$ dictionary. Use both functions $df2()$ and $c_df()$ to define a Python function $MIM(docs, a, b)$ to calculate the MIM value for terms a and b .

Solution:

(1)

Let $N = |D|$, then we have

$$P(a) = df(D, a) / N, P(b) = df(D, b) / N,$$

$$P(a, b) = df2(D, a, b) / N$$

$$\begin{aligned} \text{So, } MIM &= \log(df2(D, a, b) / N) / ((df(D, a) / N) * (df(D, b) / N)) \\ &= \log[N * df2(D, a, b) / (df(D, a) * df(D, b))] \end{aligned}$$

(2)

```
#define a function to calculate n_{ab}
def df2(docs, a, b):
    """Calculate the number of documents that include both terms."""
    nab = 0
    for id, doc in docs.items():
        if (a in doc) and (b in doc):
            nab = nab + 1
    return nab
```

(3)

```
#define a function to calculate MIM for two terms
import math
def MIM(docs, a, b):
    t_df = c_df(docs)
    na = t_df[a]
    nb = t_df[b]
    nab = df2(docs, a, b)
    _mim = math.log((len(docs)*nab/(na*nb)), 2)
    return _mim
```

Q2 (Term frequency measure)

Assume Table 1 is a training set provided by user feedback. A term's frequency measure is defined as follows:

$$tf(t_k) = \frac{\sum_{d_i \in D^+} f_{ik}}{\sum_{d_i \in D^+} \sum_{t_j \in T} f_{ij}}$$

Doc_ID	Terms	Label (Relevant)
D ₁	GERMAN, VW	0 no
D ₂	US, US, ECONOM, SPY	1 yes
D ₃	US, BILL, ECONOM, ESPIONAG	1 yes
D ₄	US, ECONOM, ESPIONAG, BILL	1 yes
D ₅	GERMAN, MAN, VW, ESPIONAG	0 no
D ₆	GERMAN, GERMAN, MAN, VW, SPY	0 no
D ₇	US, MAN, VW	0 no

Table 1. A training set from user feedback

Calculate term ESPIONAG's frequency measure, and show your calculation steps:

Solution:

$T = \{\text{US, BILL, ECONOM, ESPIONAG, GERMAN, MAN, VW, SPY}\}$

$D^+ = \{\mathbf{D_2, D_3, D_4}\}$

$D^- = \{D_1, D_5, D_6, D_7\}$

f_{ij}	US	BILL	ECONOM	ESPIONAG	GERMAN	MAN	VW	SPY	sum
D1	0	0	0	0	1	0	1	0	
D2	2	0	1	0	0	0	0	1	4
D3	1	1	1	1	0	0	0	0	4
D4	1	1	1	1	0	0	0	0	4
D5	0	0	0	1	1	1	1	0	
D6	0	0	0	0	2	1	1	1	
D7	1	0	0	0	0	1	1	0	

$$tf(\text{'ESPIONAG'}) = (0+1+1) / (4+4+4) = 2/12 = 0.167$$

Q3 (Probabilistic information filtering model)

For a given training set D (or labelled dataset, e.g., a list of lists of terms, where each list of terms denotes a document), which consists of a set of relevant documents Rel and a set of non-relevant documents $Nonrel$; BM25 based IF model uses the following equation to calculate a term's weight:

$$w_5(t) = \log \frac{\frac{r(t) + 0.5}{R - r(t) + 0.5}}{\frac{n(t) - r(t) + 0.5}{(N - n(t)) - (R - r(t)) + 0.5}}$$

Provide equations to calculate N , R , $r(t)$ and $n(t)$, where an equation is a math formula or a python definition.

Solution:

$N = |D|$, or $N = \text{len}(D)$,

which is the total number of documents in the training set D ;

$R = |\text{Rel}|$, or $R = \text{len}(\text{Rel})$, which is the number of relevant documents for a topic (or a user information need);

$n(t) = |\{d \in D, t \in d\}|$ or $n(t) = \text{len}([d \text{ for } d \text{ in } D \text{ if } t \text{ in } d])$, which is the number of documents that contain term t ; and

$r(t) = |\{d \in \text{Rel}, t \in d\}|$ or $r(t) = \text{len}([d \text{ for } d \text{ in } \text{Rel} \text{ if } t \text{ in } d])$, which is the number of relevant documents that contain term t .

You may test your python definitions:

```
>>> D = [['t1', 't2'], ['t3', 't2'], ['t1', 't2', 't5', 't4'], ['t3', 't6', 't4']]
>>> len(D)
4
>>> Rel = [['t1', 't2', 't5', 't4'], ['t3', 't6', 't4']]
>>> Nonrel = [['t1', 't2'], ['t3', 't2']]
>>> len(Rel)
2
>>> len([d for d in D if 't1' in d])
2
>>> len([d for d in Rel if 't1' in d])
1
```

Q4 Which of the following descriptions is wrong? and justify your answer.

- (1) Large language models (LLMs) can naturally generate human-readable explanations for recommendations, thereby improving system transparency and interpretability.
- (2) LLM-based information filtering does not require labelled training data, as well-designed prompts can effectively guide the model's behaviour.

- (3) Retrieval-Augmented Generation (RAG) is an AI framework that enhances LLMs by retrieving relevant information from external knowledge bases, leading to more accurate and factually grounded outputs.
- (4) LLMs such as GPT-4 have demonstrated strong capabilities in natural language understanding, and their effectiveness in real-world news recommendation systems has been fully established.
- (5) Chain-of-Thought (CoT) prompting is a method to improve LLMs by prompting them to generate step-by-step reasoning before giving a final answer.

Solution: (4)

LLMs such as GPT-4 have demonstrated strong capabilities in natural language understanding; however, their effectiveness in real-world news recommendation systems has yet to be fully established.